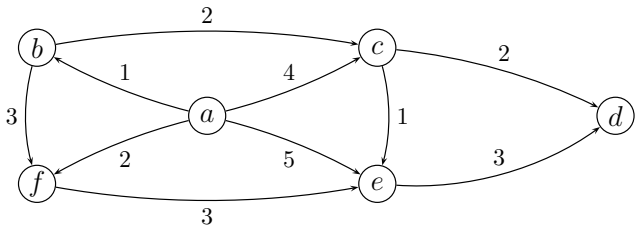
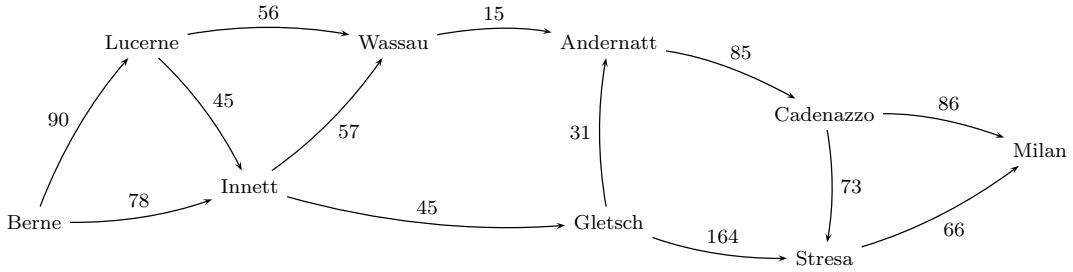
Exercice I

Appliquer l'algorithme de Bellman au graphe ci-dessous pour déterminer une a-arborescence de chemins de poids minimums.



Exercice II

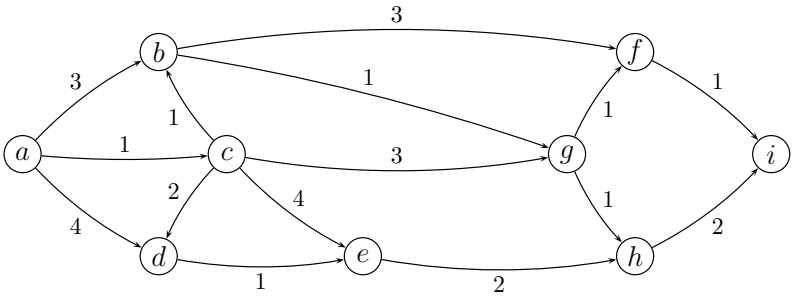
Une entreprise de transport Suissexport, dont le siège est à Berne, doit effectuer de fréquentes livraisons à Milan. Vu la fréquence et vu le coût de ces livraisons, l'entreprise désire déterminer l'itinéraire le plus court de Berne à Milan.



Les poids des arcs sont des distances exprimées en kilomètres. Déterminer le plus court chemin de Berne à Milan en utilisant l'algorithme de Bellman.

Exercice III

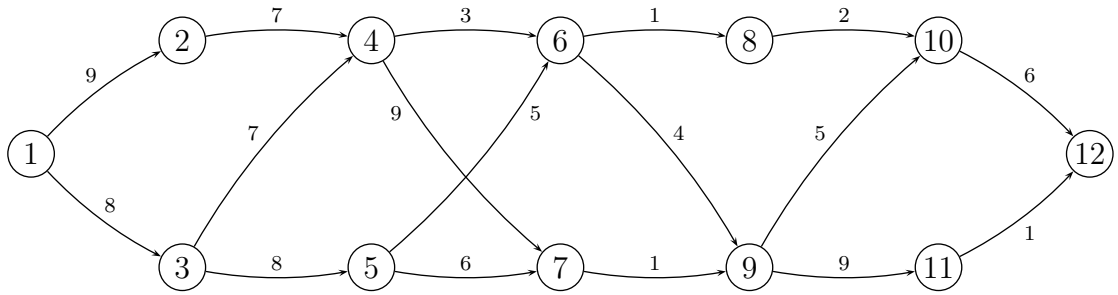
Appliquer l'algorithme de Bellman au graphe ci-dessous pour déterminer une c-arborescence de chemins de poids minimums.



Exercice IV

Partant de Moscou, Michel Strogoff, courrier du Tsar, devait rejoindre Irkoutsz. Avant de partir, il avait consulté une voyante qui lui dit entre autre chose : « après Kazan, méfiez-vous du ciel, à Omsk attention aux tartares, dans Tomsk attention aux yeux, après Tomsk méfiez-vous de l'eau et, surtout, prenez garde partout au grand brun avec des bottes noires ». Strogoff avait ainsi reporté sur une carte, pour chaque liaison entre deux villes, les chances de réussite exprimées par un nombre compris entre 1 et 10.

Les numéros des villes sont les suivantes : Moscou (1), Kazan (2), Penza (3), Perm (4), Oufa (5), Tobolosk (6), Novo-Saimsk (7), Tara (8), Omsk (9), Tomsk (10), Semipa-Latinsk (11), Irkoutsz (12).



1. Ignorant le calcul des probabilités, il avait alors choisi son itinéraire en maximisant la somme globale des chances. Quel algorithme permet de calculer un tel itinéraire ?
2. En déduire la probabilité pour que Strogoff réussisse.
3. Quel aurait été son itinéraire si Strogoff avait connu les principes du calcul des probabilités ?

Algorithme de Bellman

données : un graphe pondéré sans circuit $G = (S, A, P)$ et un sommet r de G

résultats : une r-arborescence de cpm représentée par la table des pères $\mathcal{A}$ et la table des potentiels Π

```
proc Bellman(G, r)
    C ← Fifo()
    pour tout s ∈ S faire
        marque[s] ← δ⁻(s)
        si marque[s] = 0 alors
            C.enQueue(s)
        fin si
    fin pour
    tant que non C.isEmpty() faire
        s ← C.deQueue()
        examiner(s)
        pour tout x ∈ V⁺(s) faire
            marque[x] ← marque[x] − 1
            si marque[x] = 0 alors
                C.enQueue(x)
            fin si
        fin pour
    fin tq
fin proc
```

```
proc examiner(s)
    si s = r alors
        Π[s] ← 0
        A[s] ← none
    sinon
        Π[s] ← ∞
        A[s] ← s
        pour tout x ∈ V⁻(s) faire
            si Π[x] + P(x, s) < Π[s] alors
                Π[s] ← Π[x] + P(x, s)
                A[s] ← x
            fin si
        fin pour
    fin si
fin proc
```